

```

import pandas as pd
import numpy as np
import warnings, os, json
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_predict
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, f1_score, roc_auc_score)
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from imblearn.over_sampling import SMOTE
from xgboost import XGBClassifier
import joblib

# — Optional deep learning (TF/Keras) —————
try:
    import tensorflow as tf
    from tensorflow import keras
    from tensorflow.keras import layers, callbacks
    TF_AVAILABLE = True
    tf.random.set_seed(42)
except ImportError:
    TF_AVAILABLE = False
    print("TensorFlow not available - running ML models only.")

np.random.seed(42)

```

```

CSV_PATH = "/content/features_with_labels_fix.csv"
OUTPUT_DIR = "/content/outputs"
os.makedirs(OUTPUT_DIR, exist_ok=True)

df = pd.read_csv(CSV_PATH)
print(f"Dataset: {df.shape[0]} tiles × {df.shape[1]} columns")

LABEL_COLS = ['houses', 'roads', 'ponds', 'Electric Poles']
LABEL_VALUES = {'houses': 1, 'roads': 1, 'ponds': 1, 'Electric Poles': 1}
ID_COL = 'filename'
FEATURE_COLS = [c for c in df.columns if c not in LABEL_COLS + [ID_COL]]

# Convert labels to binary (presence = 1, absence = 0)
for col, val in LABEL_VALUES.items():
    df[col + '_bin'] = (df[col] == val).astype(int)

BIN_LABEL_COLS = [c + '_bin' for c in LABEL_COLS]

X = df[FEATURE_COLS].values
y = df[BIN_LABEL_COLS].values # shape (N, 4)
image_ids = df[ID_COL].values

print(f"Features: {X.shape[1]}")
print("Label distribution (binary):")
for i, lbl in enumerate(LABEL_COLS):
    pos = y[:, i].sum()
    print(f" {lbl:16s}: {pos} present / {len(y)-pos} absent")

```

```

Dataset: 2424 tiles × 277 columns
Features: 272
Label distribution (binary):
houses      : 1796 present / 628 absent
roads       : 1433 present / 991 absent
ponds       : 193 present / 2231 absent
Electric Poles : 238 present / 2186 absent

```

```

X_train, X_test, y_train, y_test, id_train, id_test = train_test_split(
    X, y, image_ids, test_size=0.40, random_state=42, stratify=y[:, 0]
)
print(f"\nTrain: {len(X_train)} | Test: {len(X_test)}")

```

Train: 1454 | Test: 970

```
imputer = SimpleImputer(strategy='median')
scaler = StandardScaler()

X_train_s = scaler.fit_transform(imputer.fit_transform(X_train))
X_test_s = scaler.transform(imputer.transform(X_test))
```

```
print("\n" + "="*60)
print("Training ML Ensemble (RF + GBM + LR voting) per label ...")
print("="*60)

ml_results = {}
ml_classifiers = {}

for i, lbl in enumerate(LABEL_COLS):
    yt_train = y_train[:, i]
    yt_test = y_test[:, i]

    # Check if the label has only one class in training data
    if len(np.unique(yt_train)) < 2:
        print(f" [{lbl:16s}] Only one class found in training data. Skipping model training and predicting 'absent'.")
        train_pred = np.zeros_like(yt_train)
        test_pred = np.zeros_like(yt_test)

        train_acc = accuracy_score(yt_train, train_pred) * 100
        test_acc = accuracy_score(yt_test, test_pred) * 100

        # F1 score is not meaningful for a single class; set to 0
        test_f1 = 0.0

        ml_results[lbl] = {
            'train_acc': round(train_acc, 2),
            'test_acc': round(test_acc, 2),
            'f1': round(test_f1, 4),
            'report': "Only one class present, no meaningful classification report."
        }
        # Store a dummy classifier that always predicts 0
        class DummyZeroPredictor:
            def predict(self, X): return np.zeros(len(X), dtype=int)
            def predict_proba(self, X): return np.zeros((len(X), 2), dtype=float)
        ml_classifiers[lbl] = DummyZeroPredictor()

        print(f" [{lbl:16s}] Train={train_acc:.1f}% Test={test_acc:.1f}% F1={test_f1:.3f}")
    else:
        # Individual models
        rf = RandomForestClassifier(n_estimators=200, max_depth=10,
                                   min_samples_leaf=10, max_features='sqrt', class_weight='balanced',
                                   random_state=42, n_jobs=-1)
        gbm = GradientBoostingClassifier(n_estimators=100, learning_rate=0.05,
                                         max_depth=2, subsample=0.8, random_state=42)
        lr = LogisticRegression(C=0.3, class_weight='balanced',
                                max_iter=1000, random_state=42)

        # Soft-voting ensemble
        vc = VotingClassifier(estimators=[('rf', rf), ('gbm', gbm), ('lr', lr)],
                              voting='soft', weights=[2, 2, 1] )
        vc.fit(X_train_s, yt_train)

        train_pred = vc.predict(X_train_s)
        test_pred = vc.predict(X_test_s)

        train_acc = accuracy_score(yt_train, train_pred) * 100
        test_acc = accuracy_score(yt_test, test_pred) * 100
        test_f1 = f1_score(yt_test, test_pred, average='weighted')

        ml_results[lbl] = {
            'train_acc': round(train_acc, 2),
            'test_acc': round(test_acc, 2),
            'f1': round(test_f1, 4),
            'report': classification_report(yt_test, test_pred,
                                           target_names=['absent', 'present'])
```

```

    }
    ml_classifiers[lbl] = vc
    print(f" [{lbl:16s}] Train={train_acc:.1f}% Test={test_acc:.1f}% F1={test_f1:.3f}")

# Save the models, scaler, and feature columns
print("\nSaving models and artifacts to MODEL_DIR...")
joblib.dump(scaler, os.path.join(MODEL_DIR, "scaler.pkl"))
for lbl, model in ml_classifiers.items():
    joblib.dump(model, os.path.join(MODEL_DIR, f"model_{lbl}.pkl"))
with open(os.path.join(MODEL_DIR, "feature_columns.json"), 'w') as fh:
    json.dump(FEATURE_COLS, fh)
print(f"All models and artifacts saved to {MODEL_DIR}")

```

```

=====
Training ML Ensemble (RF + GBM + LR voting) per label ...
=====
[houses      ] Train=94.8% Test=89.4% F1=0.894
[roads       ] Train=87.6% Test=75.1% F1=0.749
[ponds       ] Train=97.5% Test=94.5% F1=0.938
[Electric Poles ] Train=95.0% Test=88.8% F1=0.883

Saving models and artifacts to MODEL_DIR...
All models and artifacts saved to /content

```

```

import os, json, joblib
import numpy as np
import pandas as pd
import cv2
import geopandas as gpd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

from shapely.geometry import box, LineString, Point

```

```

def extract_features(tile_bgr):
    """
    Input : tile_bgr - numpy uint8 (256x256x3) BGR (as returned by cv2)
    Output: dict with exactly the 272 keys from features_with_labels_fix.csv
    """
    f = {}
    H, W = tile_bgr.shape[:2]

    # --- colour conversions ---
    gray = cv2.cvtColor(tile_bgr, cv2.COLOR_BGR2GRAY).astype(np.float32)
    gray_u = gray.astype(np.uint8)
    rgb = cv2.cvtColor(tile_bgr, cv2.COLOR_BGR2RGB).astype(np.float32)
    hsv = cv2.cvtColor(tile_bgr, cv2.COLOR_BGR2HSV).astype(np.float32)
    lab = cv2.cvtColor(tile_bgr, cv2.COLOR_BGR2LAB).astype(np.float32)
    R, G, B = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]

    # =====
    # 1. DWT (Haar, 3 levels)
    # =====
    coeffs = pywt.wavedec2(gray, 'haar', level=3)
    # coeffs[0] = approx (LL3); coeffs[1..3] = (LH,HL,HH) at levels 3>1
    # Training uses l1=finest, l3=coarsest -> coeffs[3], coeffs[2], coeffs[1]
    level_map = {1: coeffs[3], 2: coeffs[2], 3: coeffs[1]}
    for lvl, detail in level_map.items():
        for sub_idx, sub in enumerate(detail):
            f[f'dwt_l{lvl}_sub{sub_idx}_energy'] = float(np.sum(sub**2))
            f[f'dwt_l{lvl}_sub{sub_idx}_std'] = float(np.std(sub))

    # =====
    # 2. FFT features
    # =====
    fspec = np.abs(fftshift(fft2(gray)))
    fspec += 1e-10
    f['fft_log_energy'] = float(np.sum(np.log(fspec**2)))
    total_e = np.sum(fspec**2)
    cy, cx = H//2, W//2
    r_cut = min(cy, cx)//2
    Y, X = np.ogrid[:H, :W]
    mask_hi = ((Y-cy)**2 + (X-cx)**2) > r_cut**2
    f['fft_high_freq_ratio'] = float(np.sum(fspec[mask_hi]**2) / (total_e+1e-10))
    gy, gx = np.meshgrid(np.arange(H), np.arange(W), indexing='ij')

```

```

dist = np.sqrt((gy-cy)**2 + (gx-cx)**2) + 1e-10
f['fft_spectral_centroid'] = float(np.sum(dist * fspec) / np.sum(fspec))

# =====
# 3. Radon transform (road / linear structure detection)
# =====
angles = np.linspace(0, 180, 36, endpoint=False)
sinogram = radon(gray, theta=angles, circle=False)
col_vars = np.var(sinogram, axis=0)
f['radon_max_var'] = float(np.max(col_vars))
f['radon_angle_peak'] = float(angles[np.argmax(col_vars)])
f['radon_var_std'] = float(np.std(col_vars))

# =====
# 4. Gabor filters (4 frequencies x 4 orientations)
# =====
gabor_freqs = [0.1, 0.2, 0.3, 0.4]
gabor_thetas = [0, np.pi/4, np.pi/2, 3*np.pi/4]
theta_names = ['t0', 't45', 't90', 't135']
for fi, freq in enumerate(gabor_freqs, 1):
    for ti, (theta, tname) in enumerate(zip(gabor_thetas, theta_names)):
        kernel = np.real(gabor_kernel(freq, theta=theta))
        resp = cv2.filter2D(gray, cv2.CV_32F, kernel)
        f[f'gabor_f{fi}_{tname}_mean'] = float(np.mean(resp))
        f[f'gabor_f{fi}_{tname}_std'] = float(np.std(resp))

# =====
# 5. Spectral entropy
# =====
ps = (fspec**2).ravel()
ps /= ps.sum() + 1e-10
f['spectral_entropy'] = float(-np.sum(ps * np.log(ps + 1e-10)))

# =====
# 6. Phase congruency (approximated via multi-scale Laplacian energy)
# =====
for sc, ksize in zip([1, 2, 4], [3, 5, 9]):
    lap = cv2.Laplacian(gray_u, cv2.CV_64F, ksize=ksize)
    f[f'phase_cong_scale{sc}_energy'] = float(np.sum(lap**2))

# =====
# 7. Steerable pyramid (4 orientations via oriented Gaussian derivatives)
# =====
for ang, aname in zip([0, 45, 90, 135], ['0deg', '45deg', '90deg', '135deg']):
    k = gabor_kernel(0.25, theta=np.deg2rad(ang))
    resp = cv2.filter2D(gray, cv2.CV_32F, np.real(k))
    f[f'steer_pyr_{aname}_energy'] = float(np.sum(resp**2))

# =====
# 8. Curvelet-proxy (multi-scale DWT energy histograms)
# =====
for sc, detail in zip([1, 2, 4], [coeffs[3], coeffs[2], coeffs[1]]):
    combined = np.concatenate([d.ravel() for d in detail])
    hist, _ = np.histogram(combined, bins=8, density=True)
    for bi, v in enumerate(hist):
        f[f'curvelet_s{sc}_bin{bi}'] = float(v)

# =====
# 9. Cepstrum features
# =====
cep = np.real(np.fft.ifft2(np.log(np.abs(fft2(gray)) + 1e-10)))
f['cepstrum_mean'] = float(np.mean(cep))
f['cepstrum_peak'] = float(np.max(np.abs(cep)))
f['cepstrum_std'] = float(np.std(cep))

# =====
# 10. Band-pass energies (DWT sub-band energy bands)
# =====
all_coeffs = pywt.wavedec2(gray, 'haar', level=5)
band_pairs = [(1,3),(3,7),(7,15),(15,31),(31,63)]
for (lo, hi), (c1, c2) in zip(band_pairs,
                               [(all_coeffs[5], all_coeffs[4]),
                                (all_coeffs[4], all_coeffs[3]),
                                (all_coeffs[3], all_coeffs[2]),
                                (all_coeffs[2], all_coeffs[1]),
                                (all_coeffs[1], all_coeffs[0])]):

```

```

e = sum(np.sum(s**2) for s in (c1 if isinstance(c1,tuple) else (c1,)))
f[f'bandpass_{lo}_{hi}_energy'] = float(e)

# =====
# 11. Spectral indices (NDVI proxy, NDWI, NDBI, etc.)
# =====
eps = 1e-6
nir_proxy = G # Green as NIR proxy
f['ndvi_proxy'] = float(np.mean((nir_proxy - R) / (nir_proxy + R + eps)))
f['ndwi_proxy'] = float(np.mean((G - nir_proxy) / (G + nir_proxy + eps)))
f['ndbi_proxy'] = float(np.mean((B - G) / (B + G + eps)))
f['blue_red_ratio'] = float(np.mean(B / (R + eps)))
f['water_index'] = float(np.sum((G > 100) & (B > 100) & (R < 80)))
shadow = 1.0 - (R + G + B) / (3*255 + eps)
f['shadow_index_mean'] = float(np.mean(shadow))
f['shadow_index_std'] = float(np.std(shadow))

# =====
# 12. HSV statistics
# =====
f['hsv_h_mean'] = float(np.mean(hsv[:,0]))
f['hsv_h_std'] = float(np.std(hsv[:,0]))
f['hsv_s_mean'] = float(np.mean(hsv[:,1]))
f['hsv_s_std'] = float(np.std(hsv[:,1]))
f['hsv_v_mean'] = float(np.mean(hsv[:,2]))
f['hsv_v_std'] = float(np.std(hsv[:,2]))

# =====
# 13. LAB statistics
# =====
f['lab_l_mean'] = float(np.mean(lab[:,0]))
f['lab_l_std'] = float(np.std(lab[:,0]))
f['lab_a_mean'] = float(np.mean(lab[:,1]))
f['lab_a_std'] = float(np.std(lab[:,1]))
f['lab_b_mean'] = float(np.mean(lab[:,2]))
f['lab_b_std'] = float(np.std(lab[:,2]))

# =====
# 14. HSV hue histogram (8 bins)
# =====
hue_hist, _ = np.histogram(hsv[:,0].ravel(), bins=8, range=(0,180), density=True)
for i, v in enumerate(hue_hist):
    f[f'hsv_hue_hist_{i}'] = float(v)

# =====
# 15. LAB-b skewness + colour entropy
# =====
f['lab_b_skew'] = float(skew(lab[:,2].ravel()))
for ch, chname in zip([B, G, R], ['b', 'g', 'r']):
    hist, _ = np.histogram(ch.ravel(), bins=256, range=(0,256))
    hist = hist / (hist.sum() + 1e-10)
    f[f'color_entropy_{chname}'] = float(-np.sum(hist * np.log(hist + 1e-10)))

# =====
# 16. Dominant colours (KMeans k=3)
# =====
pixels = tile_bgr.reshape(-1, 3).astype(np.float32)
km = KMeans(n_clusters=3, random_state=0, n_init=3, max_iter=50)
km.fit(pixels)
centers = km.cluster_centers_.astype(int)
# sort by cluster size so order is deterministic
counts = np.bincount(km.labels_, minlength=3)
order = np.argsort(-counts)
for rank, idx in enumerate(order):
    b_v, g_v, r_v = centers[idx]
    f[f'dom_color_{rank}_b'] = float(b_v)
    f[f'dom_color_{rank}_g'] = float(g_v)
    f[f'dom_color_{rank}_r'] = float(r_v)

# =====
# 17. PCA variance of colour channels
# =====
pca = PCA(n_components=3)
pca.fit(pixels)
for i, v in enumerate(pca.explained_variance_ratio_):
    f[f'pca_var_{i}'] = float(v)

```

```

# =====
# 18. Built-up density & specular ratio
# =====
# bright pixels with low saturation → rooftops / concrete
bright = (hsv[:, :, 2] > 180) & (hsv[:, :, 1] < 60)
lbl_arr, n_regions = nd_label(bright)
f['buildup_density'] = float(np.sum(bright) / bright.size)
f['buildup_region_count'] = float(n_regions)
f['specular_ratio'] = float(np.sum(gray > 230) / gray.size)

# =====
# 19. GLCM (Haralick)
# =====
gray_q = (gray_u // 32).astype(np.uint8) # 8 levels
glcm = graycomatrix(gray_q, distances=[1],
                    angles=[0, np.pi/4, np.pi/2],
                    levels=8, symmetric=True, normed=True)
for prop in ['contrast', 'dissimilarity', 'homogeneity', 'energy', 'correlation', 'ASM']:
    v = graycoprops(glcm, prop)
    f[f'glcm_{prop}_mean'] = float(np.mean(v))
    f[f'glcm_{prop}_std'] = float(np.std(v))

# =====
# 20. Tamura features
# =====
# Coarseness: mean of block-averaged pixel differences
for bs in [2, 4, 8, 16]:
    h2, w2 = H // bs, W // bs
    blk = gray[:h2*bs, :w2*bs].reshape(h2, bs, w2, bs).mean(axis=(1,3))
    f[f'tamura_coarse_bs{bs}'] = float(np.mean(np.abs(np.diff(blk, axis=1))))
# Contrast
mu4 = np.mean((gray - np.mean(gray))**4)
sig2 = np.var(gray) + 1e-10
f['tamura_contrast'] = float(np.std(gray) / (mu4/sig2**2 + 1e-10)**0.25)
# Directionality (edge orientation histogram entropy)
Gx = cv2.Sobel(gray_u, cv2.CV_64F, 1, 0, ksize=3)
Gy = cv2.Sobel(gray_u, cv2.CV_64F, 0, 1, ksize=3)
angles_px = np.arctan2(Gy, Gx + 1e-10)
ah, _ = np.histogram(angles_px.ravel(), bins=16, range=(-np.pi, np.pi), density=True)
f['tamura_directionality'] = float(scipy_entropy(ah + 1e-10))

# =====
# 21. LBP (3 radii)
# =====
for radius in [1, 2, 3]:
    lbp = local_binary_pattern(gray_u, P=8*radius, R=radius, method='uniform')
    hist_lbp, _ = np.histogram(lbp.ravel(), bins=8*radius+2,
                             range=(0, 8*radius+2), density=True)
    f[f'lbp_r{radius}_uniformity'] = float(hist_lbp[0] + hist_lbp[-1])
    f[f'lbp_r{radius}_entropy'] = float(scipy_entropy(hist_lbp + 1e-10))

# =====
# 22. Local variance at multiple window sizes
# =====
for ws in [8, 16, 32]:
    kernel = np.ones((ws, ws), np.float32) / (ws*ws)
    mu = cv2.filter2D(gray, cv2.CV_32F, kernel)
    mu2 = cv2.filter2D(gray**2, cv2.CV_32F, kernel)
    loc_var = np.abs(mu2 - mu**2)
    f[f'local_var_ws{ws}_mean'] = float(np.mean(loc_var))
    f[f'local_var_ws{ws}_std'] = float(np.std(loc_var))

# Local entropy (8-bit histogram)
hist_g, _ = np.histogram(gray_u.ravel(), bins=256, range=(0,256))
hist_g = hist_g / (hist_g.sum() + 1e-10)
f['local_entropy'] = float(-np.sum(hist_g * np.log(hist_g + 1e-10)))

# =====
# 23. Multi-scale Laplacian variance
# =====
for sc in [1, 2, 4, 8]:
    blurred = cv2.GaussianBlur(gray_u, (0,0), sc)
    lap = cv2.Laplacian(blurred, cv2.CV_64F)
    f[f'ms_lap_var_{sc}'] = float(np.var(lap))

```

```

# =====
# 24. Morphological opening residuals
# =====
for r in [3, 5, 9]:
    se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (r, r))
    opened = cv2.morphologyEx(gray_u, cv2.MORPH_OPEN, se)
    residual = gray.astype(float) - opened.astype(float)
    f[f'morph_open_{r}_residual_mean'] = float(np.mean(residual))
    f[f'morph_open_{r}_residual_std'] = float(np.std(residual))

# =====
# 25. Hu moments
# =====
moments = cv2.moments(gray_u)
hu_mom = cv2.HuMoments(moments).flatten()
for i, v in enumerate(hu_mom):
    sign = -1 if v < 0 else 1
    f[f'hu_{i}'] = float(sign * np.log10(abs(v) + 1e-10))

# =====
# 26. Contour shape features
# =====
_, thresh = cv2.threshold(gray_u, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
if contours:
    largest = max(contours, key=cv2.contourArea)
    area = cv2.contourArea(largest) + 1e-6
    perim = cv2.arcLength(largest, True) + 1e-6
    hull = cv2.convexHull(largest)
    hull_a = cv2.contourArea(hull) + 1e-6
    x,y,w,h = cv2.boundingRect(largest)
    f['contour_compactness'] = float(perim**2 / (4*np.pi*area))
    f['bbox_aspect_ratio'] = float(w / (h + 1e-6))
    f['extent'] = float(area / (w*h + 1e-6))
    f['circularity'] = float(4*np.pi*area / (perim**2))
    f['convex_hull_ratio'] = float(area / hull_a)
    f['contour_solidity'] = float(area / hull_a)
    f['contour_perim_area_ratio'] = float(perim / area)
    f['largest_contour_area'] = float(area)
else:
    for k in ['contour_compactness', 'bbox_aspect_ratio', 'extent', 'circularity',
             'convex_hull_ratio', 'contour_solidity', 'contour_perim_area_ratio',
             'largest_contour_area']:
        f[k] = 0.0
f['num_contours'] = float(len(contours))
f['mean_contour_area'] = float(np.mean([cv2.contourArea(c) for c in contours])
                               if contours else 0.0)

# =====
# 27. Skeleton features
# =====
bin_img = (gray_u > 127).astype(np.uint8)
skel = skeletonize(bin_img).astype(np.uint8)
f['skeleton_length'] = float(np.sum(skel))
# Branch points: neighbours == 3 or 4; endpoints: neighbours == 1
kernel_bp = np.ones((3,3), np.uint8)
neighbour_count = cv2.filter2D(skel.astype(np.float32), -1, kernel_bp.astype(np.float32)) - skel
f['skeleton_branch_points'] = float(np.sum((skel==1) & (neighbour_count >= 3)))
f['skeleton_endpoints'] = float(np.sum((skel==1) & (neighbour_count == 1)))

# =====
# 28. Distance transform
# =====
dist_t = cv2.distanceTransform(bin_img, cv2.DIST_L2, 5)
f['dist_transform_max'] = float(np.max(dist_t))
f['dist_transform_mean'] = float(np.mean(dist_t))

# =====
# 29. Hough line features
# =====
edges_c = cv2.Canny(gray_u, 50, 150)
lines = cv2.HoughLinesP(edges_c, 1, np.pi/180, threshold=30,
                        minLineLength=20, maxLineGap=10)
if lines is not None:
    lines = lines.reshape(-1, 4)
    lengths = np.sqrt((lines[:,2]-lines[:,0])**2 + (lines[:,3]-lines[:,1])**2)

```

```

    angles_l = np.arctan2(lines[:,3]-lines[:,1], lines[:,2]-lines[:,0]+1e-10)
    f['line_count'] = float(len(lines))
    f['avg_line_length'] = float(np.mean(lengths))
    f['hough_dominant_angle'] = float(np.degrees(angles_l[np.argmax(lengths)]))
    f['hough_angle_std'] = float(np.degrees(np.std(angles_l)))
else:
    f['line_count'] = f['avg_line_length'] = 0.0
    f['hough_dominant_angle'] = f['hough_angle_std'] = 0.0

# =====
# 30. Stroke Width Transform (approximate via distance transform on edges)
# =====
swt_map = cv2.distanceTransform((edges_c == 0).astype(np.uint8), cv2.DIST_L2, 5)
f['swt_mean'] = float(np.mean(swt_map))
f['swt_std'] = float(np.std(swt_map))
f['swt_max'] = float(np.max(swt_map))

# =====
# 31. Edge orientation histogram (9 bins + 8-bin copy)
# =====
mag = np.hypot(Gx, Gy)
mask_e = mag > np.percentile(mag, 50)
ori = np.arctan2(Gy, Gx + 1e-10)[mask_e]
for nbins, prefix in [(9, 'edge_orient'), (8, 'orient_hist')]:
    hist_o, _ = np.histogram(ori, bins=nbins,
                             range=(-np.pi, np.pi), density=True)
    for i, v in enumerate(hist_o):
        f[f'{prefix}_{i}'] = float(v)

# =====
# 32. Frangi vesselness (road / waterway)
# =====
fr = frangi(gray_u.astype(float) / 255.0)
f['frangi_mean'] = float(np.mean(fr))
f['frangi_max'] = float(np.max(fr))
f['frangi_std'] = float(np.std(fr))

# =====
# 33. Region properties
# =====
lbl_img, n_regs = nd_label(thresh)
props = regionprops(lbl_img)
f['region_count'] = float(n_regs)
f['region_area_mean'] = float(np.mean([p.area for p in props]) if props else 0.0)
f['region_perim_mean'] = float(np.mean([p.perimeter for p in props]) if props else 0.0)
f['region_compact_mean'] = float(np.mean(
    [p.perimeter**2 / (4*np.pi*p.area + 1e-6) for p in props if p.area > 0]
) if props else 0.0)

# =====
# 34. HOG (32 features)
# =====
hog_feat = hog(gray_u, orientations=8, pixels_per_cell=(32,32),
               cells_per_block=(2,2), visualize=False, feature_vector=True)
hog_feat = hog_feat[:32] if len(hog_feat) >= 32 else \
    np.pad(hog_feat, (0, 32-len(hog_feat)))
for i, v in enumerate(hog_feat):
    f[f'hog_{i}'] = float(v)

# =====
# 35. LoG blob + MSER (training columns)
# =====
blobs = blob_log(gray_u, max_sigma=30, num_sigma=10, threshold=0.1)
blob_areas = [np.pi * b[2]**2 for b in blobs]
f['blob_count'] = float(len(blob_areas))
f['blob_area_mean'] = float(np.mean(blob_areas) if blob_areas else 0.0)

mser = cv2.MSER_create()
regions, _ = mser.detectRegions(gray_u)
areas_m = [cv2.contourArea(np.array(r)) for r in regions if len(r) > 2]
f['mser_count'] = float(len(areas_m))
f['mser_area_mean'] = float(np.mean(areas_m) if areas_m else 0.0)

return f # 272 keys – exactly matching training CSV

```

```
# Paths
IMAGE_PATH = "/content/image_129.png"
MODEL_DIR = "/content"
OUTPUT_DIR = "/content/outputs"

# Tile settings
TILE_SIZE = 256
OVERLAP = 32
BLACK_THRESH = 15
MIN_VALID_RATIO = 0.40

# Labels
LABELS = ['houses', 'roads', 'ponds', 'Electric Poles']

LAYER_NAME = {
    'houses': 'houses',
    'roads': 'roads',
    'ponds': 'ponds',
    'Electric Poles': 'electric_poles'
}
```

```
def make_polygon(r, c, ts):
    return box(float(c), float(r), float(c+ts), float(r+ts))

def make_road(r, c, ts):
    mid = float(r + ts//2)
    return LineString([(float(c), mid), (float(c+ts), mid)])

def make_point(r, c, ts):
    return Point(float(c + ts//2), float(r + ts//2))

GEOM_FN = {
    'houses': make_polygon,
    'ponds': make_polygon,
    'roads': make_road,
    'Electric Poles': make_point,
}
```

```
def make_polygon(r, c, ts):
    return box(float(c), float(r), float(c+ts), float(r+ts))

def make_road(r, c, ts):
    mid = float(r + ts//2)
    return LineString([(float(c), mid), (float(c+ts), mid)])

def make_point(r, c, ts):
    return Point(float(c + ts//2), float(r + ts//2))

GEOM_FN = {
    'houses': make_polygon,
    'ponds': make_polygon,
    'roads': make_road,
    'Electric Poles': make_point,
}
```

```
def save_debug_map(img_bgr, tile_size, overlap,
                  accepted, rejected, pred_lookup, out_path):

    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    fig, ax = plt.subplots(figsize=(16, 16))
    ax.imshow(img_rgb)

    # Rejected tiles
    for (r, c) in rejected:
        ax.add_patch(mpatches.Rectangle(
            (c,r), tile_size, tile_size,
            lw=0.4, edgecolor='red', facecolor='red', alpha=0.20))

    # Accepted tiles
    for (r, c) in accepted:
        any_pred = any(pred_lookup.get((r,c,lbl), 0) for lbl in LABELS)
        col = 'lime' if any_pred else 'deepskyblue'
        alp = 0.35 if any_pred else 0.08
```

```

        ax.add_patch(mpatches.Rectangle(
            (c,r), tile_size, tile_size,
            lw=0.5, edgecolor=col, facecolor=col, alpha=alp))

ax.legend(handles=[
    mpatches.Patch(color='lime', label='Detected'),
    mpatches.Patch(color='deepskyblue', label='No feature'),
    mpatches.Patch(color='red', label='Skipped'),
], loc='lower right')

ax.set_title('Tile Debug Map')
ax.axis('off')
plt.savefig(out_path, dpi=150)
plt.close()

print(f"Debug map → {out_path}")

```

```

def run(image_path=IMAGE_PATH,
        model_dir=MODEL_DIR,
        output_dir=OUTPUT_DIR,
        tile_size=TILE_SIZE,
        overlap=OVERLAP):

    os.makedirs(output_dir, exist_ok=True)

    out_gpkg = os.path.join(output_dir, "village_prediction.gpkg")
    out_csv = os.path.join(output_dir, "tile_predictions.csv")
    out_debug = os.path.join(output_dir, "tile_debug.png")

    print("\n=== Village Prediction Pipeline ===")

    # 1. Load models
    scaler = joblib.load(os.path.join(model_dir, "scaler.pkl"))

    models = {
        lbl: joblib.load(os.path.join(model_dir, f"model_{lbl}.pkl"))
        for lbl in LABELS
    }

    with open(os.path.join(model_dir, "feature_columns.json")) as f:
        feature_columns = json.load(f)

    print("Models loaded ✅")

    # 2. Load image
    img_bgr = cv2.imread(image_path)
    if img_bgr is None:
        raise FileNotFoundError(image_path)

    H, W = img_bgr.shape[:2]
    print(f"Image size: {W}x{H}")

    # 3. Tile processing
    step = tile_size - overlap

    geom_store = {lbl: [] for lbl in LABELS}
    tile_records = []

    accepted, rejected = [], []
    pred_lookup = {}

    for r in range(0, H, step):
        for c in range(0, W, step):

            patch = img_bgr[r:r+tile_size, c:c+tile_size]

            # Padding
            ph = tile_size - patch.shape[0]
            pw = tile_size - patch.shape[1]

            if ph > 0 or pw > 0:
                patch = np.pad(patch, ((0,ph),(0,pw),(0,0)))

            gray = cv2.cvtColor(patch, cv2.COLOR_BGR2GRAY)

            # Filter black tiles

```

```

valid_ratio = np.sum(gray > BLACK_THRESH) / gray.size

if valid_ratio < MIN_VALID_RATIO:
    rejected.append((r,c))
    continue

accepted.append((r,c))

# FEATURE EXTRACTION (YOU MUST DEFINE THIS FUNCTION)
feat = extract_features(patch)

X = np.array([[feat.get(col, 0) for col in feature_columns]])
X_scaled = scaler.transform(X)

preds = {
    lbl: int(models[lbl].predict(X_scaled)[0])
    for lbl in LABELS
}

tile_records.append({
    'row': r,
    'col': c,
    **preds
})

for lbl in LABELS:
    pred_lookup[(r,c,lbl)] = preds[lbl]

    if preds[lbl] == 1:
        geom_store[lbl].append({
            'row': r,
            'col': c,
            'geometry': GEOM_FN[lbl](r,c,tile_size)
        })

print("Tiles processed ✅")

# 4. Save outputs
if os.path.exists(out_gpkg):
    os.remove(out_gpkg)

for lbl in LABELS:
    if len(geom_store[lbl]) == 0:
        continue

    gdf = gpd.GeoDataFrame(geom_store[lbl], geometry='geometry')
    gdf.set_crs("EPSG:4326", inplace=True)

    gdf.to_file(out_gpkg, layer=LAYER_NAME[lbl], driver="GPKG")

pd.DataFrame(tile_records).to_csv(out_csv, index=False)

save_debug_map(img_bgr, tile_size, overlap,
               accepted, rejected, pred_lookup, out_debug)

print("\nOutputs saved ✅")
print(out_gpkg, out_csv, out_debug)

return out_gpkg

```

run()

```

=== Village Prediction Pipeline ===
Models loaded ✅
Image size: 512x512
Tiles processed ✅
Debug map → /content/outputs/tile_debug.png

Outputs saved ✅
/content/outputs/village_prediction.gpkg /content/outputs/tile_predictions.csv /content/outputs/tile_debug.png
'/content/outputs/village_prediction.gpkg'

```

Start coding or [generate](#) with AI.

